

Customising OpenSSL for the Real World

The OpenSSL project is a robust collective effort that seeks to develop a commercial grade, fullfeatured toolkit implementation of SSL and TLS. This article is a detailed introduction to OpenSSL.

A part from introducing you to OpenSSL, this article explores the scale of its usage and, hence, the need to customise it based on real world requirements. It then goes into the details of the engine and BIO interfaces, and how external hardware or software can be hooked into these interfaces to override the default software implementation and make OpenSSL more relevant in the real world.

This article is meant for casual readers interested in knowing about OpenSSL as well as open source hobbyists who would like to go under the hood to look at OpenSSL source code to see how they can make it work the way they want it to. It may also be helpful to serious developers trying to explore ways to improve their SSL traffic performance.

What is SSL/TLS and why is it important?

Secure Sockets Layer and its new avatar, Transport Layer Security, are cryptographic algorithms that are used to establish a secure channel of communication between two end points. Three versions of SSL—1.0, 2.0 and 3.0—were published up to the late 90s, the last being the biggest overhaul of the protocol. Subsequently, TLS 1.0 was defined in RFC 2246 as a small upgrade over SSL v3.0. As there is very little difference between the two, the terms SSL and TLS are usually used interchangeably.

As the names suggest, SSL or TLS are mechanisms to secure connections and sit on top of the transport layer. They involve arriving at a shared session key using the initial handshake protocol. The initial handshake messages are primarily based on CPU-intensive public key cryptographic algorithms. Once the session is established, data is sent encrypted with a shared session key using much faster symmetric key algorithms, which are decrypted on the other end.

SSL/TLS is one of the most pervasive cryptographic protocols used in the market today. Most popular application layer protocols like HTTP, SMTP and FTP use it for a secure lower layer. Most commonly used applications like online banking, Gmail, Facebook, Twitter, etc. use HTTPS and hence TLS/SSL as a secure channel to communicate personal and confidential information. These servers handle millions of TLS/SSL sessions concurrently.

As per Sandvine's Global Internet Phenomenon Report – 2H 2012 (see References), by 2018, SSL traffic is expected to increase 16-fold over its 2012 levels, putting much more pressure on these servers to handle concurrent sessions. Figure 1 demonstrates the trend in SSL traffic growth.